

Entrega 2, N1 - Códigos do Savino

Grupo EcoNexus

Gustavo Kenji Matsune

Maria Eduarda da Silva Souza

Pedro Henrique Dutra Silva

Stefany Anne Apaza de Freitas

RESUMO DO JOGO

O jogo se passa em uma terra indígena ameaçada por constantes invasões de garimpeiros e criminosos, que colocam em risco a integridade da aldeia, da floresta e da vida animal. Ao iniciar a jornada, o jogador assume o papel de um guerreiro da tribo, responsável por proteger seu território. Sua missão principal é patrulhar a região e sabotar os equipamentos dos invasores como: tratores, escavadoras e caminhões, agindo com estratégia e furtividade para não ser detectado.

Código da Câmera

```
using UnityEngine;

public class CameraI : MonoBehaviour
{
    public float velocidade = 5.0f;
    public float sensibilidadeMouse = 2.0f;
    public Transform cameraTransform;

    private float rotacaoX = 0f;

    void Start()
    {
        Cursor.lockState = CursorLockMode.Locked;

        if (cameraTransform == null)
        {
            Debug.Log("Ei! Você esqueceu de arrastar a Câmera para o campo Camera Transform no script!");
        }
    }
}
```

```

    }
}

void Update()
{
    float mouseX = Input.GetAxis("Mouse X") *
sensibilidadeMouse;
    transform.Rotate(Vector3.up * mouseX);

    float mouseY = Input.GetAxis("Mouse Y") *
sensibilidadeMouse;
    rotacaoX -= mouseY;
    rotacaoX = Mathf.Clamp(rotacaoX, -90f, 90f);
    Debug.Log("Rotação Atual: " + rotacaoX);

    if (cameraTransform != null)
    {
        cameraTransform.localEulerAngles = new
Vector3(rotacaoX, 0, 0);
    }

    float moverFrente = Input.GetAxis("Vertical");
    float moverLado = Input.GetAxis("Horizontal");

    Vector3 direcao = transform.forward * moverFrente +
transform.right * moverLado;
    direcao.y = 0;

    if (direcao.magnitude > 0.1f)
    {
        transform.Translate(direcao.normalized * velocidade *
Time.deltaTime, Space.World);
    }
}
}

```

Código de Spawnar o Equipamento aleatoriamente

```

using System.Collections.Generic;
using UnityEngine;

```

```

public class Equipment : MonoBehaviour
{

    public Quaternion RotateValue;
    public float equipmentRotation = -90f;

    public GameObject equipmentPrefab;

    private List<GameObject> equipmentList = new
List<GameObject>();

    private List<Vector3> equipmentPositions = new
List<Vector3>();

    void Start()
    {
        RotateValue = Quaternion.AngleAxis(equipmentRotation,
Vector3.right);
        SpawnEquipment();
    }

    private void SpawnEquipment()
    {
        float minDistance = 20f;

        int equipmentQuantity = Random.Range(2, 7);
        for (int i = 0; i < equipmentQuantity; i++)
        {
            Vector3 randomSpawnPosition = new
Vector3(Random.Range(-220, -110), -0.854f, Random.Range(-40, 70));

            int attempt = 0;
            bool validPosition = false;

            while (!validPosition && attempt < 10)
            {
                randomSpawnPosition = new
Vector3(Random.Range(-220, -100), -0.854f, Random.Range(-35, 75));

                validPosition = true;
            }
        }
    }
}

```

```

        foreach (Vector3 pos in equipmentPositions)
        {
            if (Vector3.Distance(pos, randomSpawnPosition)
< minDistance)
            {
                validPosition = false;
                break;
            }
        }

        attempt++;

    }

    if (equipmentPositions.Contains(randomSpawnPosition))
    {
        Debug.Log("Sem posi'o livre");
return;
    }

    equipmentPositions.Add(randomSpawnPosition); /

    GameObject newEquipment = Instantiate(equipmentPrefab,
randomSpawnPosition, RotateValue);

    equipmentList.Add(newEquipment);
}
}
}

```

Código da Barra de Vida

```

using UnityEngine;
using UnityEngine.UI;

public class HealthBar : MonoBehaviour
{
    [SerializeField] private Image imageHealthBar;

```

```

private Transform myCamera;

private void Awake()
{
    myCamera = Camera.main.transform;
}

void Update()
{
    transform.LookAt(transform.position + myCamera.forward);
}

public void alterHealthBar(int currentLife, int maxLife)
{
    imageHealthBar.fillAmount = (float)currentLife / maxLife;
}
}

```

Código de Vida do Equipamento

```

using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;

public class EquipmentLife : MonoBehaviour
{
    [SerializeField] private int maxLife = 200;
    [SerializeField] private HealthBar healthBar;

    [SerializeField] private int pointsValue = 50;

    [SerializeField] private int currentLife;

    public static int totalScore = 0;

    public static int totalEquipments;

    void Start()
    {

```

```

        totalEquipments++;

        currentLife = maxLife;
        healthBar.alterHealthBar(currentLife, maxLife);
    }

    private void OnTriggerStay(Collider other)
    {
        if (other.CompareTag("Player") && Input.GetKey(KeyCode.Mouse0))
        {
            TakeDamage(5);
        }
    }

    private void TakeDamage(int damage)    {
        currentLife -= damage;

        if (currentLife < 0)
            currentLife = 0;

        healthBar.alterHealthBar(currentLife, maxLife);

        if (currentLife <= 0) // se a vida for igual ou menor que 0,
destroi o obj
        {
            AwardPoints();

            totalEquipments--;

            if (totalEquipments <= 0)
            {
                SceneManager.LoadScene("GameOver");
            }

            Destroy(gameObject);
        }
    }

    private void AwardPoints()
    {
        totalScore += pointsValue;
        Debug.Log("Pontuação Atual: " + totalScore);

        ScoreManager.instance.UpdateScoreUI();
    }

```

```
}
```

Código de Vida do Jogador

```
using UnityEngine;
public class Life : MonoBehaviour
{
    private int currentLife;
    private int maxLife = 100;
    [SerializeField] private HealthBar healthBar;

    void Start()
    {
        currentLife = maxLife;

        healthBar.alterHealthBar(currentLife, maxLife);
    }

    void Update()
    {
        if (Input.GetButtonDown("Fire1"))
        {
            TakeDamage();
        }
    }

    private void TakeDamage()
    {
        currentLife -= 10;
        healthBar.alterHealthBar(currentLife, maxLife);
    }
}
```

Código de Inimigo Persegue

```
using UnityEngine;

public class InimigoPersegue : MonoBehaviour
{
    public Life PlayerLifeScript;
    public Transform player;
    public float velocidade = 3f;
```

```

private void Start()
{
    player = GameObject.FindGameObjectWithTag("Player").transform;
}
void Update()
{
    Vector3 direcao = (PlayerRef.Player.position -
transform.position).normalized;

    transform.position += direcao * velocidade * Time.deltaTime;

    transform.LookAt(player);
}
private void OnTriggerStay(Collider other)
{
    if (other.CompareTag("Player")){
        PlayerLifeScript = PlayerRef.PlayerL;
        PlayerLifeScript.TakeDamage();
    }
}
}

```

Código de Inimigo Foge

```

using UnityEngine;
using UnityEngine.AI;

public class InimigoFoge : MonoBehaviour
{

    public float raioFuga = 5f;
    public float velocidadeFuga = 4f;
    public float raioSeguranca = 10f;

    public Transform jogador;

    private NavMeshAgent agente;
    private bool estahFugindo = false;

    void Start()
    {

```



```

    if (jogador == null)
    {
        Debug.LogError("Jogador NÃO encontrado!");
    }
    agente = GetComponent<NavMeshAgent>();

    if (agente == null)
    {
        Debug.LogError("NavMeshAgent não encontrado no inimigo");
        return;
    }

    agente.speed = velocidadeFuga;

    if (jogador == null)
    {
        GameObject obj = GameObject.FindWithTag("Player");
        if (obj != null)
            jogador = obj.transform;
        else
            Debug.LogWarning("Jogador não encontrado");
    }
}

void Update()
{
    Vector3 teste = new Vector3(14, 0, -29);
    agente.SetDestination(teste);

    if (jogador == null || agente == null) return;

    float distancia = Vector3.Distance(transform.position,
jogador.position);

    if (distancia < raioFuga)
    {
        estahFugindo = true;
        Fugir();
    }
    else if (estahFugindo && distancia > raioSeguranca)
    {
        estahFugindo = false;
        agente.ResetPath();
    }
}

```

```

    void Fugir()
    {
        Vector3 direcaoFuga = (transform.position -
jogador.position).normalized;

        Vector3 destinoFuga = transform.position + direcaoFuga *
raioSeguranca;

        NavMeshHit hit;
        if (NavMesh.SamplePosition(destinoFuga, out hit, 2f,
NavMesh.AllAreas))
        {
            agente.SetDestination(hit.position);
        }
    }

    void OnDrawGizmosSelected()
    {
        Gizmos.color = Color.red;
        Gizmos.DrawWireSphere(transform.position, raioFuga);

        Gizmos.color = Color.green;
        Gizmos.DrawWireSphere(transform.position, raioSeguranca);
    }
}

```

Código de Movimentação do Jogador

```

using UnityEngine;

public class MovimentoJogador : MonoBehaviour
{
    public float velocidade = 4f;
    public float velocidadeCorrida = 5f;

    void Update()
    {
        float x = Input.GetAxis("Horizontal");
        float z = Input.GetAxis("Vertical");

        Vector3 movimento = new Vector3(x, 0, z);

        if (Input.GetKey(KeyCode.LeftShift))
        {
            transform.Translate(movimento * velocidadeCorrida *

```

```

Time.deltaTime);
    }
    else
    {
        transform.Translate(movimento * velocidade *
Time.deltaTime);
    }
}
}

```

Código da Pontuação

```

using TMPro;
using UnityEngine;

public class ScoreManager : MonoBehaviour
{
    public static ScoreManager instance;

    [SerializeField] private TMP_Text scoreText;

    private void Awake()
    {
        instance = this;
    }

    void Start()
    {
        UpdateScoreUI();
    }

    public void UpdateScoreUI()
    {
        scoreText.text = "Pontuação: " + EquipmentLife.totalScore;
    }
}

```

Código do Menu

```

using UnityEngine;
using UnityEngine.SceneManagement;

```

```

public class Menu : MonoBehaviour
{
    [SerializeField] private string sceneName;
    [SerializeField] private GameObject menuPanel;

    public void Play()
    {
        SceneManager.LoadScene(sceneName);
    }

    public void Exit()
    {
        Debug.Log("Saindo");
        Application.Quit();
    }
}

```

Código do Cronômetro

```

using UnityEngine;
using TMPro;

public class Cronometro : MonoBehaviour
{
    [Header("Configuracoes de Tempo")]
    public float tempoInicial = 150f;

    [Header("Componentes de UI")]
    public TextMeshProUGUI textoTempo;

    private float tempoAtual;
    private bool cronometroRodando;

    void Start()
    {
        tempoAtual = tempoInicial;
        cronometroRodando = true;
        AtualizarTexto();
    }

    void Update()
    {
        if (!cronometroRodando) return;
    }
}

```

```

        tempoAtual -= Time.deltaTime;

        if (tempoAtual <= 0f)
        {
            tempoAtual = 0f;
            cronometroRodando = false;
            TempoEsgotado();
        }

        AtualizarTexto();
    }

    void AtualizarTexto()
    {
        int minutos = Mathf.FloorToInt(tempoAtual / 60f);
        int segundos = Mathf.FloorToInt(tempoAtual % 60f);
        textoTempo.text = string.Format("{0}:{1:00}", minutos,
segundos);
    }

    void TempoEsgotado()
    {
        Debug.Log("O tempo acabou!");
        textoTempo.text = "0:00";
    }
}

```

Código de Referência do Jogador (para registrar que ele foi spawnado)

```

using UnityEngine;
public class PlayerRef : MonoBehaviour
{
    public static Life PlayerL;
    public static Transform Player;

    void Awake()
    {
        PlayerL = GetComponent<Life>();
        Player = transform;
    }
}

```

Código de Spawnar o Player (SpawnPoint)

```
using UnityEngine;

public class SpawnPointPlayer : MonoBehaviour
{
    public GameObject playerPrefab;

    void Awake()
    {
        SpawnPlayer();
    }

    public void SpawnPlayer()
    {
        GameObject NewPlayer = Instantiate(playerPrefab,
transform.position, Quaternion.identity);
        NewPlayer.tag = "Player";
    }
}
```